

How Much to Trust a Learned Model: Safe Adaptive Dynamic Programming for Supervising Residual-Augmented Quadrotor MPC

MD Ashfakul Karim Kausik

October 2026

Abstract

Learned residual models are routinely added to quadrotor model predictive control (MPC) to compensate for model mismatch. In my earlier study, the same accurate residual helped, did nothing, or degraded tracking depending on the mismatch. The relevant question is therefore not whether the residual is accurate, but how much of it the controller should use at each moment. This proposal treats that question as a constrained optimal control problem. A scale $\alpha_k \in [0, 1]$ blends the learned correction into the reference of an unchanged nominal MPC. An analytic hard layer bounds α_k with three closed-form quantities: a realizability ratio that addresses actuator saturation, a small-gain bound that addresses the feedback path closed by the correction, and a control barrier condition that addresses state limits. Within that bound, an action-dependent adaptive dynamic programming (ADP) agent learns the trust policy, using model uncertainty and measured prediction error as information. Because $\alpha \equiv 0$ reproduces the nominal controller, it serves as the admissible initial policy, and every later policy must pass a statistical acceptance test against it. This gives a route to a guarantee, in expectation over the mismatches under study, that the learned supervisor does not degrade the controller it supervises. The engineering components are scoped for a master's program in simulation, while the stability and non-degradation results form the theoretical core of the work.

1 Introduction and Motivation

MPC is the standard tool for quadrotor trajectory tracking because it plans over a horizon while respecting actuator limits [1]. Its accuracy depends on its internal model, and mass change, rotor drag, motor lag and actuation delay all cause mismatch. Learned residual models are the common remedy, but my earlier study showed that the same accurate residual can help, do nothing, or degrade tracking depending on the mismatch. The practical question is therefore not whether a residual is accurate, but how much of it the controller should use at each moment, and whether that decision can be learned with a guarantee that it does not degrade the nominal design.

Two mechanisms can make an accurate correction harmful. The correction can demand more input than the actuators can deliver, and it can close an additional feedback path around a loop whose stability margin the mismatch has already reduced. Model uncertainty, the signal most often used to gate a learned term, measures neither. This proposal therefore bounds the use of the correction with quantities that measure both mechanisms directly, and learns the trust level inside that bound.

2 Background and Related Work

This section reviews the prior work on which the proposal builds. It covers four areas: learned residual models in quadrotor MPC, safety layers and run-time assurance, methods that gate or blend a learned correction, and adaptive dynamic programming with safe policy improvement. It closes with a summary table and a statement of the research gap that the proposal addresses.

2.1 Learned residuals in quadrotor MPC

Augmenting a nominal model with a learned term was formalized with robustness guarantees by Aswani et al. [2] and demonstrated on board a quadrotor [3]. Recent work brought the idea to agile flight: Torrente et al. [5] embed a Gaussian-process residual inside the MPC, Salzmann et al. [6] make neural-network dynamics real-time, and Saviolo and Loianno [7] review learned quadrotor dynamics. Neural-Fly [22] adapts a learned residual online for flight in strong wind. These works improve model accuracy and apply the learned term whenever it is available.

Model mismatch is also handled without a learned model. Offset-free MPC [23] rejects constant disturbances through an augmented disturbance state, and $\mathcal{L}_1$ -adaptive nonlinear MPC [24] compensates payload and aerodynamic mismatch on a real quadrotor. These are the natural non-learning baselines for any residual method, and they do not address the case where a learned correction is present and may be harmful.

2.2 Safety layers and run-time assurance

Cautious MPC [8] and safe-exploration MPC [4] propagate learned uncertainty into tightened constraints. Predictive safety filters [9] certify any proposed input by solving a second MPC problem online. Reference governors [17] scale a reference to preserve constraint admissibility, and learning reference governors [25] learn the constraint boundary from data while enforcing it. Control barrier functions [20] give closed-form, invariance-based safety conditions. The Simplex architecture [21] adds the principle of reverting to a verified baseline, and Fallback-Safe MPC [26] pairs a calibrated run-time monitor with a robust fallback plan. These methods decide whether an action is safe. They do not decide how much of a learned correction is worth using, and the filter-based methods require an online optimization.

2.3 Gating and blending a learned correction

Several works scale a learned term instead of applying it fully. Residual reinforcement learning [27] adds a learned policy to a fixed controller, and Residual MPC [30] blends a learned policy with MPC torques through a fixed scaling factor. Control regularization [28] mixes a learned policy with a control prior through a weight that trades variance against bias. CHEQ [29] adapts this weight online from the uncertainty of a critic ensemble. For quadrotors, Colombo et al. [31] suppress a Gaussian-process compensation through a smooth uncertainty gate. Iscan and Temiz [32] supervise the references of an unchanged fixed-wing autopilot with residual Q-learning, a barrier-inspired shield, and a no-op fallback. In all of these, the scale is either fixed, a hand-designed function of uncertainty, or a discrete learned action without a guarantee relative to the baseline controller. None of them uses actuator realizability as the signal.

The distinction between prediction accuracy and realizability has been stated in a different setting. Wu et al. [33] certify Koopman learning control of repetitive systems and show that a small prediction residual does not imply that the requested output correction lies in the constrained reachable set. Their test is an offline certification based on constrained least squares. It is not a run-time bound, and it is not applied to residual-augmented MPC.

2.4 Adaptive dynamic programming and safe policy improvement

Adaptive dynamic programming learns near-optimal feedback policies from data through actor-critic and policy-iteration schemes [13, 14], where convergence and stability rest on starting from an admissible policy. Berkenkamp et al. [10] gave stability guarantees for model-based safe reinforcement learning. Recent work has combined model-based reinforcement learning with high-order barrier functions for systems of high relative degree [11] and established safe learning control with joint optimality

and stability guarantees under disturbances [12]. Safe policy improvement [34] gives guarantees that a learned policy is no worse than a baseline policy. These methods learn the control policy itself. They have not been applied to the trust policy that arbitrates between a nominal controller and a learned model.

2.5 Summary and research gap

Table 1 summarizes the works closest to this proposal.

Table 1: Representative prior work and the gap each leaves relative to this proposal.

Work	What it does	Main contribution	Gap relative to this proposal
<i>Learned residuals in MPC</i>			
Aswani et al. [2]	Separates a nominal model for safety from a learned model for performance	Robustness guarantees for learning-based MPC	The learned model is always used in the cost; no run-time trust level
Torrente et al. [5]	Embeds a Gaussian-process residual in quadrotor MPC	Large tracking gains in agile flight	Residual applied unconditionally
Salzmann et al. [6]	Runs neural-network dynamics inside MPC in real time	Real-time neural MPC on hardware	Assumes the learned term helps; no fallback
O’Connell et al. [22]	Adapts a learned residual online in strong wind	Fast online adaptation with stability analysis	Adapts the residual, not the trust placed in it
<i>Mismatch handling without a learned model</i>			
Pannocchia and Rawlings [23]	Augments MPC with a disturbance state	Offset-free tracking under constant mismatch	No learned correction to supervise
Hanover et al. [24]	Adds $\mathcal{L}_1$ adaptation to nonlinear MPC	Robustness to payload and drag on a real quadrotor	No learned correction to supervise
<i>Safety layers and run-time assurance</i>			
Hewing et al. [8]	Propagates Gaussian-process uncertainty into constraints	Cautious, uncertainty-aware MPC	Tightens constraints; does not scale the correction
Wabersich and Zeilinger [9]	Filters any input through a second MPC	Safety certificate for arbitrary learned inputs	Online optimization; binary accept or modify
Garone et al. [17]	Scales a reference to keep constraints admissible	Mature reference-governor theory	Not driven by a learned correction or its realizability
Liu et al. [25]	Learns the constraint boundary for a reference governor	Constraint enforcement during and after learning	Learns constraints, not the value of a correction
Sinha et al. [26]	Monitors perception and switches to a robust fallback	End-to-end assurance with calibrated monitors	Binary switch; monitors perception, not a residual
<i>Gating and blending a learned correction</i>			
Cheng et al. [28]	Mixes a learned policy with a control prior	Variance reduction with stability discussion	Mixing weight is fixed or scheduled
Cramer et al. [29]	Adapts the mixing weight from critic uncertainty	Safer and faster hybrid learning	Weight is a rule, not optimized; no guarantee
Jeon et al. [30]	Blends a learned policy with MPC torques	Residual MPC on a humanoid	Fixed scale; no guarantee relative to the MPC
Colombo et al. [31]	Gates a learned quadrotor compensation by uncertainty	Probabilistic tracking bound with a smooth gate	Hand-designed gate; uncertainty only; no realizability
Iskan and Temiz [32]	Residual Q-learning on the references of a fixed-wing autopilot	Autopilot-preserving supervision with a shield	Discrete actions; no baseline guarantee
Wu et al. [33]	Certifies whether a requested correction is reachable under input constraints	Separates prediction accuracy from realizability	Offline certification; not a run-time bound for MPC

Work	What it does	Main contribution	Gap relative to this proposal
<i>Adaptive dynamic programming and safe learning</i>			
Liu and Wei [14]	Policy iteration for discrete-time nonlinear systems	Convergence from an admissible initial policy	Learns the control law, not a trust policy
Wang et al. [12]	Safe learning control under disturbances	Joint optimality and stability guarantees	Learns the control law, not a trust policy
Laroche et al. [34]	Improves a policy while staying close to a baseline	Guarantee of no degradation relative to the baseline	Offline, discrete setting; not applied to control supervision

Three gaps remain. First, existing gates for learned corrections are driven by uncertainty, although uncertainty does not indicate whether the actuators can deliver the correction. Second, where realizability has been studied, it is an offline certificate and not a run-time, solver-free bound computed from quantities the MPC already produces. Third, where the scale of a learned correction is adapted, it follows a hand-designed rule and carries no guarantee relative to the nominal controller. To the author’s knowledge, no existing method combines (i) a closed-form realizability bound on the use of a learned residual, evaluated at run time from the predicted inputs of the MPC, with (ii) a trust policy that is optimized inside that bound and initialized at the nominal controller, with non-degradation relative to that controller as an explicit target. This proposal addresses both by casting the trust decision as a safe ADP problem whose admissible initial policy is the nominal MPC itself.

3 Preliminary Work and Current Status

My earlier study, presented at IEEE iCONEECT 2026 [19] (code available in the [uav-mpc-learning](#) repository), injected a learned residual as a feedforward term into a quadrotor MPC in MuJoCo. The same residual improved tracking under some mismatches, had no measurable effect under others, and degraded tracking under motor lag and delay near the stability boundary. The study established the pattern but did not explain it.

A first-principles rebuild is under way in the [uav-mpc-safe-residual](#) repository. The rigid-body dynamics, the actuator mixer, the constraint polytope, the hover equilibrium and the linearization about hover are derived and documented. This analysis already suggests an explanation. A mass error is a constant bias on the vertical channel, which has relative degree two. Drag is a smooth, state-dependent force that the nominal controller largely rejects by itself. Motor lag and delay leave the equilibrium unchanged and instead remove phase margin from the lateral channel, which is a chain of four integrators from body torque to position. These observations are recorded as predictions before any experiment and motivate the first research question. The controller, the residual network and the supervisor are not yet implemented.

4 Research Questions and Hypotheses

1. **Which mechanism causes the harm, and does it survive a safer injection point?** In the proposed architecture the correction shifts only the reference supplied to the MPC, so it can no longer bypass the actuator constraints. Two mechanisms can still degrade tracking. The first is *saturation*: the shifted reference demands more input than the rotors can deliver within the prediction horizon. The second is *loop gain*: the correction depends on the measured state, so it closes an additional feedback path around a loop whose stability margin motor lag and delay have already reduced. It is hypothesized that a mass mismatch is corrected at any trust level, that a drag correction changes little, and that the degradation under motor lag and delay is caused mainly by the second mechanism and appears

before any actuator saturates. The hypothesis is tested with an oracle correction, computed from the true mismatch, before any network is trained.

2. **Do the analytic bounds predict help and harm better than model uncertainty does?** It is hypothesized that the realizability ratio r_H and the small-gain bound α_{sg} (Sec. 5.2) separate helpful from harmful corrections better than the ensemble variance. A correction is labeled helpful or harmful by a paired rollout: the same state, trajectory and random seed are simulated once with $\alpha = 0$ and once with $\alpha = 1$, and the tracking costs are compared. Predictive power is measured as classification AUC over the regime map.
3. **Can ADP learn the trust policy?** It is hypothesized that a learned soft layer achieves lower tracking cost than the nominal controller, the best fixed α , and an uncertainty-only gate, with one policy across all mismatch types, including mismatch magnitudes and combinations that were not seen in training.
4. **What can be guaranteed?** Three results are targeted (Sec. 5.4): full transmission of the correction when $\alpha \leq r_H$, closed-loop stability and constraint satisfaction for any soft policy, and non-degradation of the learned supervisor relative to the nominal controller.

5 Proposed Methodology

This section presents the proposed supervisory framework. It first defines the mismatch model and the way in which the learned correction enters the control loop. It then defines the analytic hard layer, formulates the soft layer as an ADP problem with a safe policy improvement step, and states the theoretical results that the work aims to establish.

5.1 Mismatch Model and Architecture

Motor lag and actuation delay are dynamic effects. They cannot be written as a static function of the current state and input. The true plant is therefore modeled on an augmented state:

$$z_{k+1} = F_\theta(z_k, u_k), \quad z_k = (x_k, \xi_k), \quad \theta \in \Theta, \quad (1)$$

Here $x_k \in \mathbb{R}^n$ denotes the rigid-body state, $u_k \in \mathbb{R}^m$ denotes the vector of commanded rotor thrusts, ξ_k denotes the actuator state, which contains the delivered rotor thrusts and the buffered delayed commands, and θ collects the mismatch parameters, namely the mass ratio, the drag coefficients, the motor time constant and the delay. The set Θ contains the mismatches under study. The nominal model $x_{k+1} = f(x_k, u_k)$ used by the MPC corresponds to the nominal parameter θ_0 with ideal actuators. The mismatch observed in the rigid-body state is the one-step prediction error:

$$d_k = x_{k+1} - f(x_k, u_k), \quad (2)$$

Because d_k depends on ξ_k , it depends on the recent input history and not only on (x_k, u_k) . A deep ensemble [15] therefore predicts $\hat{d}_k$ and its variance σ_k^2 from the window $w_k = (x_k, u_{k-L}, \dots, u_{k-1})$ of length L . The prediction is converted into a reference shift:

$$\Delta r_k = \text{sat}_{\bar{\Delta}}(\Gamma \hat{d}_k), \quad \|\Delta r_k\| \leq \bar{\Delta}, \quad (3)$$

Here Γ denotes a fixed matrix, derived from the nominal model, that maps a predicted acceleration error to the reference offset that cancels it at steady state, and $\bar{\Delta}$ denotes the saturation level. The map Γ is validated with the true mismatch d_k in place of $\hat{d}_k$ before any network is trained.

The nominal MPC, $u_k = \kappa(x_k, \tilde{r}_k)$, is never modified. It receives the shaped reference:

$$\tilde{r}_k = r_k + \alpha_k \Delta r_k, \quad \alpha_k \in [0, \bar{\alpha}_k], \quad (4)$$

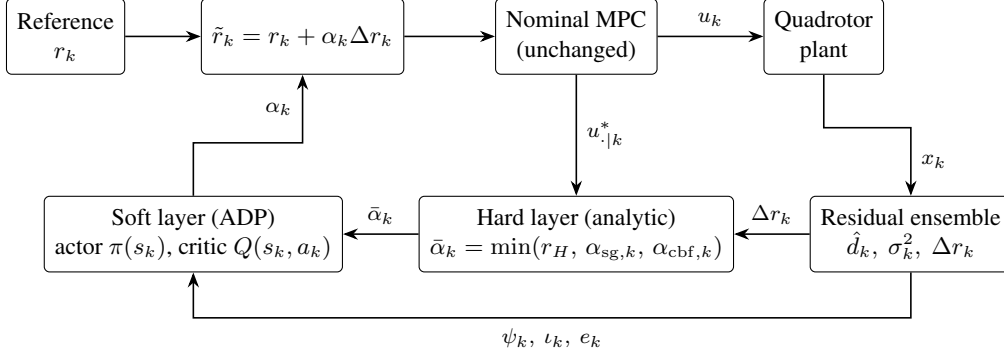


Figure 1: Proposed architecture. All safety claims rest on the analytic hard layer. The learned soft layer can only choose within $[0, \bar{\alpha}_k]$, and the uncertainty signals enter the soft layer as information, not as a safety bound.

Here r_k denotes the original reference trajectory, $\tilde{r}_k$ denotes the reference passed to the MPC, α_k denotes the trust scale selected by the supervisor, and $\bar{\alpha}_k$ denotes the upper bound on that scale. The bound $\bar{\alpha}_k$ is computed by an analytic hard layer, and the value of α_k within the bound is selected by a learned soft layer (Fig. 1). When $\alpha_k = 0$, the system reduces exactly to the nominal baseline.

5.2 The Hard Layer: Closed-Form Bounds without a Second Optimization

The hard layer combines three closed-form bounds. Each addresses a different failure mechanism: the realizability ratio addresses saturation, the small-gain bound addresses loop gain, and the barrier bound addresses state limits. Only quantities that can be bounded analytically enter the hard layer. Estimated quantities, such as the ensemble variance, are passed to the soft layer instead.

Realizability ratio. At each step, the MPC already returns its predicted inputs $u_{j|k}^*$ for $j = 0, \dots, N-1$, which are subject to box limits $[u_{\min}, u_{\max}]$ and a rate limit $\delta_{\max}$. Let $S_j = \partial u_j^* / \partial \tilde{r}$ denote the sensitivity of the unconstrained MPC law to its reference. The correction requests the additional input $\delta u_j = S_j \Delta r_k$. The realizability ratio is defined as the largest fraction of this request that the actuators can absorb:

$$r_H = \max \left\{ \beta \in [0, 1] : u_{\min} \leq u_{j|k}^* + \beta \delta u_j \leq u_{\max}, \right. \\ \left. \|(u_{j|k}^* - u_{j-1|k}^*) + \beta(\delta u_j - \delta u_{j-1})\|_{\infty} \leq \delta_{\max} \quad \forall j \right\}, \quad (5)$$

Here N denotes the prediction horizon, $u_{j|k}^*$ denotes the input predicted at step k for j steps ahead, δu_j denotes the additional input requested by the correction, and β denotes the admissible fraction of that request. Because the constraints are boxes and the request is affine in β , r_H reduces to a ratio test, the minimum of $O(Nm)$ divisions for m rotors, and requires no solver. If a rotor is already at a limit and the correction pushes it further, then $r_H = 0$. The sensitivity S_j is first computed offline about hover. A second version uses the sensitivity at the current MPC solution, and the exact value, which requires a linear program, serves as offline ground truth for both.

Small-gain bound. The reference shift depends on the measured state through the ensemble, so the correction closes a feedback path: a state deviation changes Δr_k , the scaled shift $\alpha_k \Delta r_k$ moves the reference, and the reference moves the state. Let L_{Δ} denote a Lipschitz constant of the map from the window w_k to Δr_k , and let $\gamma_{cl}(\theta)$ denote the gain of the nominal closed loop, for plant parameter θ , from a reference perturbation to the resulting state deviation. By the small-gain theorem [36, 37], the

interconnection remains stable when $\alpha_k L_\Delta \gamma_{\text{cl}}(\theta) < 1$. The bound is therefore:

$$\alpha_{\text{sg},k} = \min\left(1, \frac{1 - \epsilon_{\text{sg}}}{L_\Delta \bar{\gamma}_k}\right), \quad \bar{\gamma}_k = \sup_{\theta \in \Theta_k} \gamma_{\text{cl}}(\theta), \quad (6)$$

Here $\epsilon_{\text{sg}} \in (0, 1)$ denotes a design margin and $\Theta_k \subseteq \Theta$ denotes the set of plant parameters that is consistent with the data up to step k . In the first version $\Theta_k = \Theta$, so the bound is a constant that is computed offline. The gain $\gamma_{\text{cl}}(\theta)$ is computed offline over a grid of θ from the linearized closed loop, which consists of the unconstrained MPC law and the augmented plant (1). The constant L_Δ is enforced on the ensemble by spectral normalization.

The bound has a direct physical reading. A constant mismatch such as a mass error gives a correction that barely depends on the state, so L_Δ is small and $\alpha_{\text{sg},k} = 1$. Motor lag and delay raise γ_{cl} sharply as the stability boundary is approached, so $\alpha_{\text{sg},k}$ falls toward zero there. This is the regime in which the preliminary study observed harm.

Barrier bound. Let $h(x) \geq 0$ denote a safety function encoding tilt and altitude limits. The MPC already enforces these limits on the nominal model. The barrier bound guards them against the mismatch. Under the same linearization, the one-step state prediction is affine in α , and the discrete-time barrier condition [20] is imposed on it with a robustness margin:

$$\hat{x}_{k+1}(\alpha) \approx \bar{x}_{k+1} + \alpha M_k \Delta r_k, \quad h(\hat{x}_{k+1}(\alpha)) \geq (1 - \eta) h(x_k) + \epsilon_h, \quad (7)$$

Here $\bar{x}_{k+1}$ denotes the state predicted under the nominal reference, M_k denotes the sensitivity of the next state to the reference shift, $\eta \in (0, 1]$ denotes the barrier decay rate, and $\epsilon_h \geq 0$ denotes a margin that covers the linearization and prediction errors. With $c_k = \nabla h(\bar{x}_{k+1})^\top M_k \Delta r_k$, the condition gives the closed form:

$$\alpha_{\text{cbf},k} = \begin{cases} 1, & c_k \geq 0, \\ \text{clip}\left(\frac{h(\bar{x}_{k+1}) - (1 - \eta) h(x_k) - \epsilon_h}{-c_k}, 0, 1\right), & c_k < 0. \end{cases} \quad (8)$$

Because the lateral channel has relative degree greater than one, this step will follow the high-order barrier constructions developed for model-based safe reinforcement learning [11, 12].

The three bounds are combined into the hard bound:

$$\bar{\alpha}_k = \min(r_H, \alpha_{\text{sg},k}, \alpha_{\text{cbf},k}). \quad (9)$$

5.3 The Soft Layer: The Trust Decision as an ADP Problem

The trust state is low-dimensional, and all of its components are already available in the loop:

$$s_k = (r_H, \alpha_{\text{sg},k}, \psi_k, \iota_k, \|e_k\|, \|e_k\| - \|e_{k-1}\|, \alpha_{k-1}) \in \mathbb{R}^7, \quad (10)$$

Here e_k denotes the tracking error at step k , α_{k-1} denotes the trust scale applied at the previous step, and ψ_k and ι_k denote two signals that describe the quality of the learned model:

$$\psi_k = \text{clip}\left(\frac{\sigma_{\text{max}}^2 - \sigma_k^2}{\sigma_{\text{max}}^2 - \sigma_{\text{min}}^2}, 0, 1\right), \quad \iota_k = \|d_{k-1} - \hat{d}_{k-1}\|, \quad (11)$$

Here σ_{min}^2 and σ_{max}^2 denote calibration thresholds for the ensemble variance, and ι_k denotes the realized prediction error of the ensemble at the previous step, which is measurable one step later from (2). Deep ensembles are known to underestimate uncertainty far from the training data. For this reason ψ_k is used as information for the soft layer and not as a safety bound, and it is paired with the measured error ι_k .

The action is a bounded increment $a_k \in [-\rho, \rho]$. The trust scale and the cost to be minimized are:

$$\alpha_k = \Pi_{[0, \bar{\alpha}_k]}(\alpha_{k-1} + a_k), \quad J(\pi) = \mathbb{E}_{\theta \sim P_\Theta} \left[\sum_{k \geq 0} \gamma^k (e_k^\top Q e_k + \lambda a_k^2) \right], \quad (12)$$

Here $\Pi_{[0, \bar{\alpha}_k]}$ denotes projection onto the interval $[0, \bar{\alpha}_k]$, ρ denotes the maximum rate of increase of trust, π denotes the soft-layer policy, $\gamma \in (0, 1)$ denotes the discount factor, $Q \succ 0$ denotes the tracking-error weight, $\lambda > 0$ denotes the penalty on changes in trust, and P_Θ denotes the distribution over mismatch parameters, reference trajectories and initial conditions used in training. The projection allows the hard layer to reduce α immediately, while increases are rate-limited by ρ .

ADP is proposed because the transition $s_k \mapsto s_{k+1}$ depends jointly on the plant, the unknown mismatch, the MPC and the network, and therefore has no usable closed form. It must be learned from data. Action-dependent heuristic dynamic programming, an actor–critic scheme with critic $Q(s, a)$ [13], will be trained on simulated rollouts across mismatch types and magnitudes. The problem is deliberately small, with seven states and one action, so it runs on a CPU and its behavior can be inspected.

Initial policy. Policy iteration requires an admissible initial policy [14]. In this setting such a policy is available by construction: the policy π_0 that keeps $\alpha = 0$ reproduces the nominal controller, so $J(\pi_0) = J_{\text{nom}}$.

Safe policy improvement step. The trust state (10) is a partial observation of the closed loop, since the plant state and the mismatch parameter are hidden. The classical monotone-improvement property of exact policy iteration therefore cannot be assumed. Each actor–critic update is instead treated as a *candidate*, and a candidate replaces the current policy only if it passes a statistical test, in the spirit of high-confidence and safe policy improvement [39, 34]. Let π_i denote the current policy and π' the candidate. Both are evaluated on n paired rollouts that share the same mismatch parameter, reference and random seed, which gives the cost differences $D_1, \dots, D_n$ with $D_\ell = J_\ell(\pi_i) - J_\ell(\pi')$. The candidate is accepted if and only if

$$\frac{1}{n} \sum_{\ell=1}^n D_\ell - b_n(\delta_i) > 0 \quad \text{and} \quad \hat{J}_m(\pi_0) - \hat{J}_m(\pi') - b_{n_m}(\delta_i) \geq -\epsilon_{\text{tol}} \quad \text{for every mismatch type } m, \quad (13)$$

Here $b_n(\delta_i)$ denotes the radius of an empirical Bernstein confidence bound at level δ_i , $\hat{J}_m$ denotes the empirical cost on the n_m rollouts of mismatch type m , and $\epsilon_{\text{tol}} \geq 0$ denotes a tolerance. The first condition requires a statistically significant improvement over the current policy on average. The second condition prevents an average gain from hiding a loss on one mismatch type. The confidence levels are chosen so that the levels of all tests, over all iterations and mismatch types, sum to at most δ . Because the evaluation uses complete rollouts and not the Bellman equation, the test does not require the trust state to be Markov.

5.4 Theoretical Targets

The analysis rests on three assumptions and aims at three results.

Assumption 1 (Nominal controller). *The nominal MPC is an MPC-for-tracking design [16, 38]. It is recursively feasible for any change of its reference.*

Assumption 2 (Admissible mismatch set). *For every $\theta \in \Theta$, the closed loop of the plant (1) with the nominal MPC satisfies the state and input constraints and is input-to-state stable [37] with respect to perturbations of its reference, with gain $\gamma_{\text{cl}}(\theta) \leq \bar{\gamma} < \infty$.*

Assumption 3 (Correction map). *The reference shift satisfies $\|\Delta r_k\| \leq \bar{\Delta}$ and is Lipschitz in the window w_k with constant L_Δ .*

Assumption 2 defines the scope of the claims. The set Θ contains the mismatches for which the nominal controller is itself stable. No claim is made beyond the stability boundary of the nominal loop, where the supervisor can only return to $\alpha = 0$.

Target Result 1 (Transmission of the correction). *Consider the MPC problem linearized about the current operating point. If no input constraint is active at the solution for the reference r_k and $\alpha_k \leq r_H$, then no input constraint is active at the solution for $\tilde{r}_k$, and the applied input equals the unconstrained affine law $u_k = K_x x_k + K_r \tilde{r}_k$. The correction is then transmitted in full, and the closed loop is linear in α_k .*

This result gives the realizability ratio its role in the theory. It follows from the piecewise-affine structure of linear MPC [35] and is exact for the linearized problem. For the nonlinear problem it holds to first order, and the size of the remaining error is examined numerically.

Target Result 2 (Safety for any soft policy). *Under Assumptions 1–3, for any soft policy and any $\alpha_k \in [0, \bar{\alpha}_k]$: (i) the MPC remains recursively feasible; (ii) the closed loop is input-to-state stable, with gain at most $\bar{\gamma}/\epsilon_{\text{sg}}$ from the exogenous reference variation to the tracking error; and (iii) $h(x_k) \geq 0$ is preserved whenever the linearization and prediction errors are bounded by ϵ_h .*

Part (i) follows from Assumption 1 alone and holds for any $\alpha_k \in [0, 1]$. Parts (ii) and (iii) are where the hard layer is needed: without the small-gain bound, the correction path can destabilize a loop that satisfies Assumption 2. The result holds regardless of what the learning component does, so the soft layer can be trained without compromising safety. The main difficulty is that γ_{cl} is computed for the unconstrained affine law, which is valid only where Target Result 1 applies. The two bounds are therefore coupled, and the analysis must treat them together.

Target Result 3 (Non-degradation of the learned supervisor). *Let the policies $\pi_1, \pi_2, \dots$ be produced by the acceptance test (13) starting from π_0 . With probability at least $1 - \delta$, every accepted policy satisfies $J(\pi_i) < J(\pi_{i-1}) \leq J(\pi_0) = J_{\text{nom}}$, and $J_m(\pi_i) \leq J_m(\pi_0) + \epsilon_{\text{tol}}$ for every mismatch type m .*

The guarantee is statistical. It holds in expectation over the training distribution P_Θ and inside the simulator, and it requires bounded rollout costs, which Target Result 2 provides. It does not cover mismatches outside Θ . Two extensions form the open part of the proposal: a bound on the degradation under a shift of the mismatch distribution, and an online version with convergence guarantees in the spirit of recent safe learning control results [12].

6 Empirical Evaluation: Experimental Setup, Baselines, and Evaluation Metrics

Setup. The experiments will use a quadrotor simulated in MuJoCo and an MPC-for-tracking implemented in CasADi. IPOPT will be used during development, and acados [40] will be used for the timing results, since computation time is one of the reported metrics. Four mismatch types, namely mass, rotor drag, motor lag and actuation delay, will be studied individually and in combination. The trajectories are hover, a circle, a figure-eight, and an aggressive figure-eight that operates close to the actuator limits, since the realizability ratio is informative only near saturation. A second vehicle with different inertial properties will be used to assess cross-vehicle generalization, with all thresholds expressed in normalized quantities. The final policies will also be evaluated in a second simulator with independent rotor and aerodynamic models, as a sim-to-sim check ahead of hardware.

Table 2: Baselines and the question each one answers.

#	Baseline	Question it answers
1	Nominal MPC ($\alpha \equiv 0$)	What must not be underperformed?
2	Unsupervised residual ($\alpha \equiv 1$)	What does the correction do with no supervision?
3	Best fixed α	Is a time-varying trust level needed?
4	Hard layer with a threshold rule, no learning	What does the ADP soft layer add?
5	Uncertainty-only gate [31]	Is realizability needed, or does uncertainty suffice?
6	Soft layer trained by a standard actor-critic method without the acceptance test	What does the safe improvement step add?
7	Residual inside the MPC model [5]	How much performance does reference-level injection give up?
8	Offset-free MPC [23]	Is a learned model needed for constant mismatch?
9	$\mathcal{L}_1$ -adaptive MPC [24]	Is a learned model needed at all?
10	Predictive safety filter [9]	How does a closed-form bound compare with an online optimization?
11	Reference governor [17]	How does it compare with classical reference scaling?

Baselines. The proposed supervisor will be compared against the baselines in Table 2.

Metrics. Performance will be assessed by the root-mean-square and peak tracking errors, the fraction of time the actuators are saturated, the number of constraint violations, and the per-step computation time of the supervisor on a CPU. The predictive power of r_H and α_{sg} relative to σ^2 will be assessed by the AUC for classifying helpful and harmful corrections, with labels from the paired rollouts defined in Sec. 4. Every result will be reported over at least ten random seeds with confidence intervals, and comparisons between methods will use paired tests on common seeds.

Generalization. The learned policy will be tested on mismatch magnitudes outside the training range, on combinations of mismatches that were not seen in training, and with one mismatch type held out of training in rotation.

Ablations. Four ablations are planned: removal of each hard bound in turn; removal of each component of the trust state in turn; the hover-based realizability ratio against the solution-based version and the exact linear program; and the global Lipschitz constant L_Δ against a local estimate from the ensemble Jacobian.

Stress tests. Three stress tests will complement these experiments. In the first, a mismatch is introduced during flight. In the second, the residual network is deliberately corrupted. Both examine whether the supervisor reverts to the nominal controller as predicted by Target Result 2. In the third, the mismatch is increased beyond the set Θ until the nominal loop itself becomes unstable, in order to locate and report the boundary beyond which no claim is made.

7 Timeline and Scope

The work is planned over four semesters:

- **Semester 1:** Completion of the dynamics model, validation of the simulator against the derived equations, and implementation of the MPC-for-tracking. Characterization of the effect of each mismatch, and the oracle correction test that addresses the first research question.

- **Semester 2:** Development of the residual ensemble and the reference-shift map. Implementation of the realizability ratio, the closed-loop gain computation, the small-gain bound and the barrier bound. Construction of the regime map to address the second research question, and analysis of Target Result 1. Preparation of a conference paper on the hard layer and the regime map.
- **Semester 3:** Development of the ADP soft layer with the safe policy improvement step. Implementation of the baselines, investigation of the third research question, and analysis of Target Result 2.
- **Semester 4:** Analysis of Target Result 3. Cross-vehicle validation, evaluation in the second simulator, and the stress tests. Completion of the thesis and preparation of a journal submission on the learned supervisor.

The work is scoped to simulation. The analytic hard layer and the regime map form a complete contribution if the learned soft layer does not improve on a fixed rule, and the first paper is planned around them for that reason.

8 Doctoral Direction

Three directions extend this work. The first is on-board hardware deployment. The closed-form hard layer is designed to fit embedded computational resources, and flight experiments are the necessary test of whether the guarantees obtained in simulation survive a real mismatch.

The second is an online version of the supervisor. In this proposal the small-gain bound uses a fixed parameter set and the trust policy is trained offline. Narrowing the parameter set from flight data would make the bound less conservative, and learning the trust policy online would require convergence and non-degradation guarantees that hold during learning.

The third is the multi-vehicle setting. When vehicles track cooperatively over a communication graph, one vehicle's use of an unreliable residual propagates through its neighbors' references. The natural extension is a distributed trust policy, in which each agent's trust scale α_i depends on its local bounds and on the trust states of its neighbors, posed as a cooperative optimal control problem on graphs [18] and solved with distributed safe ADP.

9 Conclusion

This proposal reframes a practical question, namely how much a controller should trust a learned model, as a safe optimal control problem. Three analytic bounds, based on actuator realizability, loop gain and barrier functions, keep the closed loop stable and within its constraints for any choice of trust. Inside those bounds, adaptive dynamic programming, initialized at the nominal controller and filtered by a statistical acceptance test, learns the trust policy. The intended outcome is a learned supervisor that is inexpensive to run, safe by construction, and shown with high confidence not to degrade the controller it supervises over the mismatches under study.

References

- [1] J. B. Rawlings, D. Q. Mayne, and M. M. Diehl, *Model Predictive Control: Theory, Computation, and Design*, 2nd ed., Nob Hill Publishing, 2017.
- [2] A. Aswani, H. Gonzalez, S. S. Sastry, and C. J. Tomlin, "Provably safe and robust learning-based model predictive control," *Automatica*, vol. 49, no. 5, 2013.
- [3] P. Bouffard, A. Aswani, and C. J. Tomlin, "Learning-based model predictive control on a quadrotor: Onboard implementation and experimental results," *Proc. IEEE ICRA*, 2012.

- [4] T. Koller, F. Berkenkamp, M. Turchetta, and A. Krause, “Learning-based model predictive control for safe exploration,” *Proc. IEEE CDC*, 2018.
- [5] G. Torrente, E. Kaufmann, P. Foehn, and D. Scaramuzza, “Data-driven MPC for quadrotors,” *IEEE Robotics and Automation Letters*, vol. 6, no. 2, 2021.
- [6] T. Salzmann, E. Kaufmann, J. Arrizabalaga, M. Pavone, D. Scaramuzza, and M. Ryll, “Real-time neural MPC: Deep learning model predictive control for quadrotors and agile robotic platforms,” *IEEE Robotics and Automation Letters*, vol. 8, no. 4, 2023.
- [7] A. Saviolo and G. Loianno, “Learning quadrotor dynamics for precise, safe, and agile flight control,” *Annual Reviews in Control*, vol. 55, 2023.
- [8] L. Hewing, J. Kabzan, and M. N. Zeilinger, “Cautious model predictive control using Gaussian process regression,” *IEEE Trans. Control Systems Technology*, vol. 28, no. 6, 2020.
- [9] K. P. Wabersich and M. N. Zeilinger, “A predictive safety filter for learning-based control of constrained nonlinear dynamical systems,” *Automatica*, vol. 129, 2021.
- [10] F. Berkenkamp, M. Turchetta, A. P. Schoellig, and A. Krause, “Safe model-based reinforcement learning with stability guarantees,” *Proc. NeurIPS*, 2017.
- [11] T. Zhang, J. Xu, and H. Zhang, “Enhancing safety in model-based reinforcement learning with high-order control barrier functions,” *Int. J. Robust and Nonlinear Control*, vol. 35, pp. 3844–3855, 2025.
- [12] X. Wang, H. Zhang, S. Wang, W. Xiao, and M. Guay, “Safe learning control with optimality and stability guarantees,” *IEEE Trans. Automatic Control*, 2026, doi: 10.1109/TAC.2026.3707508.
- [13] F. L. Lewis and D. Vrabie, “Reinforcement learning and adaptive dynamic programming for feedback control,” *IEEE Circuits and Systems Magazine*, vol. 9, no. 3, 2009.
- [14] D. Liu and Q. Wei, “Policy iteration adaptive dynamic programming algorithm for discrete-time nonlinear systems,” *IEEE Trans. Neural Networks and Learning Systems*, vol. 25, no. 3, 2014.
- [15] B. Lakshminarayanan, A. Pritzel, and C. Blundell, “Simple and scalable predictive uncertainty estimation using deep ensembles,” *Proc. NeurIPS*, 2017.
- [16] D. Limon, I. Alvarado, T. Alamo, and E. F. Camacho, “MPC for tracking piecewise constant references for constrained linear systems,” *Automatica*, vol. 44, no. 9, 2008.
- [17] E. Garone, S. Di Cairano, and I. Kolmanovsky, “Reference and command governors for systems with constraints: A survey on theory and applications,” *Automatica*, vol. 75, 2017.
- [18] H. Zhang, F. L. Lewis, and A. Das, “Optimal design for synchronization of cooperative systems: State feedback, observer and output feedback,” *IEEE Trans. Automatic Control*, vol. 56, no. 8, 2011.
- [19] M. A. K. Kausik et al., “Learning-augmented control for quadrotor MPC in MuJoCo: When a feedforward residual architecture helps, hurts, or does nothing,” to appear in *Proc. IEEE iCONECT*, 2026.
- [20] A. D. Ames, S. Coogan, M. Egerstedt, G. Notomista, K. Sreenath, and P. Tabuada, “Control barrier functions: Theory and applications,” *Proc. European Control Conference (ECC)*, 2019.
- [21] L. Sha, “Using simplicity to control complexity,” *IEEE Software*, vol. 18, no. 4, 2001.
- [22] M. O’Connell, G. Shi, X. Shi, K. Azizzadenesheli, A. Anandkumar, Y. Yue, and S.-J. Chung, “Neural-Fly enables rapid learning for agile flight in strong winds,” *Science Robotics*, vol. 7, no. 66, 2022.
- [23] G. Pannocchia and J. B. Rawlings, “Disturbance models for offset-free model-predictive control,” *AIChE Journal*, vol. 49, no. 2, 2003.
- [24] D. Hanover, P. Foehn, S. Sun, E. Kaufmann, and D. Scaramuzza, “Performance, precision, and payloads: Adaptive nonlinear MPC for quadrotors,” *IEEE Robotics and Automation Letters*, vol. 7, no. 1, 2022.
- [25] K. Liu, N. Li, I. Kolmanovsky, D. Rizzo, and A. Girard, “Safe learning reference governor: Theory and application to fuel truck rollover avoidance,” arXiv:2101.09298, 2021.
- [26] R. Sinha, E. Schmerling, and M. Pavone, “Closing the loop on runtime monitors with fallback-safe MPC,” *Proc. IEEE CDC*, 2023.
- [27] T. Johannink, S. Bahl, A. Nair, J. Luo, A. Kumar, M. Loskyll, J. A. Ojea, E. Solowjow, and S. Levine, “Residual reinforcement learning for robot control,” *Proc. IEEE ICRA*, 2019.
- [28] R. Cheng, A. Verma, G. Orosz, S. Chaudhuri, Y. Yue, and J. Burdick, “Control regularization for reduced variance reinforcement learning,” *Proc. ICML*, 2019.
- [29] E. Cramer, B. Frauenknecht, R. Sabirov, and S. Trimpe, “Contextualized hybrid ensemble Q-learning: Learning fast with control priors,” arXiv:2406.19768, 2024.
- [30] S. H. Jeon, H. J. Lee, S. Hong, and S. Kim, “Residual MPC: Blending reinforcement learning with GPU-parallelized model predictive control,” arXiv:2510.12717, 2025.

- [31] L. Colombo, T. Beckers, and J. Giribet, “Aggressiveness-aware learning-based control of quadrotor UAVs with safety guarantees,” arXiv:2602.21936, 2026.
- [32] M. Iscan and B. Temiz, “Autopilot-preserving residual Q-learning with HJB-inspired finite-action risk filtering for fixed-wing UAV command supervision,” arXiv:2606.01397, 2026.
- [33] Y. Wu, Y. Cao, and J. Cao, “Input-to-state stability certification via projection residuals for Koopman learning control of nonlinear repetitive systems,” arXiv:2607.06459, 2026.
- [34] R. Laroche, P. Trichelair, and R. Tachet des Combes, “Safe policy improvement with baseline bootstrapping,” *Proc. ICML*, 2019.
- [35] A. Bemporad, M. Morari, V. Dua, and E. N. Pistikopoulos, “The explicit linear quadratic regulator for constrained systems,” *Automatica*, vol. 38, no. 1, 2002.
- [36] Z.-P. Jiang, A. R. Teel, and L. Praly, “Small-gain theorem for ISS systems and applications,” *Mathematics of Control, Signals, and Systems*, vol. 7, 1994.
- [37] Z.-P. Jiang and Y. Wang, “Input-to-state stability for discrete-time nonlinear systems,” *Automatica*, vol. 37, no. 6, 2001.
- [38] D. Limon, A. Ferramosca, I. Alvarado, and T. Alamo, “Nonlinear MPC for tracking piece-wise constant reference signals,” *IEEE Trans. Automatic Control*, vol. 63, no. 11, 2018.
- [39] P. S. Thomas, G. Theodorou, and M. Ghavamzadeh, “High confidence policy improvement,” *Proc. ICML*, 2015.
- [40] R. Verschueren, G. Frison, D. Kouzoupis, J. Frey, N. van Duijkeren, A. Zanelli, B. Novoselnik, T. Albin, R. Quirynen, and M. Diehl, “acados: A modular open-source framework for fast embedded optimal control,” *Mathematical Programming Computation*, vol. 14, 2022.